

指针与抽象

结构体、指针与STL容器 · 10题 · C++ & Python 双语对照

小鲁已经掌握了函数和递归。但小华告诉他："真正的工程代码很少只有简单的int和数组——你需要结构体把多个数据捆在一起，需要指针让数据之间产生链接，需要标准容器处理复杂的数据组织需求。"小鲁深吸一口气："所以.....编程世界才刚刚开始展开？"

本章10道题从三个角度切入：链表基础操作（NQ096-101）——替换空格到链表合并，学会用指针/引用操作链式结构；结构体与排序（NQ102）——C++ struct vs Python tuple, lambda vs 比较函数；函数重载与数组操作（NQ103-105）——多态思维和大数组处理。学完本章，你将从"会写代码"升级到"理解编程语言的设计哲学"。

本章角色

-  小鲁（初学者）· 第一次接触指针和链表，从"为什么需要这个"到"原来如此巧妙"
-  小华（算法高手）· 用两个栈实现队列、反转链表三指针——经典面试题的活字典
-  小栋（工程师）· 用"合并两叠排序好的扑克牌"把归并思想讲得生动易懂
-  小嘉（校主精神）· 阐释抽象的力量："抽象不是复杂度，抽象是管理的艺术"

C++ vs Python 结构体/容器对比

- 结构体/类: C++ struct/class → Python class/namedtuple/dataclass
- 链表: C++指针操作 → Python用list模拟（重在学习原理）
- 栈/队列: C++ stack/queue → Python list/deque
- 函数重载: C++同名不同参 → Python鸭子类型天然多态
- 排序: C++ sort+cmp → Python sorted+lambda

NQ096 替换空格 AcWing 16

小鲁在写一个URL编码程序："网络请求中的空格必须转成%20....."他在循环里逐个字符检查。小华走过来说："这道题看似简单，但考察的是字符串构建的思维。C++中逐字符拼接用+= '%20'，Python直接用replace(' ','%20')一行搞定。注意——不是打印，而是构建一个新字符串返回。理解'构建新对象'和'原地修改'的区别，是进阶编程的重要分水岭。"

-  输入 一个字符串（可能含空格）。
-  输出 空格替换为%20后的字符串。
-  范围 $0 \leq \text{字符串长度} \leq 1000$

输入

We are happy.

输出

We%20are%20happy.

思路 字符串构建：遍历每个字符，是空格就追加%20，否则追原字符。Python: s.replace(' ', '%20')一行。C++: for(char c:str) res+=(c==' '?"%20":string(1,c))。注意构建新字符串而非原地修改。

技巧 Python的replace()返回新字符串（不可变性）。C++的std::string支持+=拼接。这道题也是理解Python字符串不可变性的好例子——所有的'修改'其实都是'创建新版'。

C++

```
class Solution {
public:
    string replaceSpaces(string &str) {
        string res;
        for (auto c : str)
            if (c == ' ') res += "%20";
            else res += c;
        return res;
    }
};
```

Python

```
# Python solution
```

NQ097 从尾到头打印链表 AcWing 17

小鲁第一次接触链表："这些节点通过next指针连在一起——只能从头沿着指针走到尾，不能反向走。那怎么才能从尾到头打印呢？"小华说："聪明的做法——先遍历链表把值存到数组里，然后翻转数组输出。Python一行arr[::-1]翻转。栈的LIFO（后进先出）特性天然适合'反向输出'——这个思维后续会用在表达式求值、括号匹配等场景。"

输入

链表节点值序列，以-1结束。

输出

从尾到头的节点值，每行一个。

范围

链表长度 ≤ 1000

输入

1 2 3 -1

输出

3
2
1

思路 先遍历链表收集值→翻转数组→输出。本质是用数组模拟栈（先入后出）。Python: arr[::-1]翻转。C++: reverse(res.begin(),res.end())。如果面试要求不用额外空间，可以递归（利用函数调用栈天然的后进先出）。

技巧 Python翻转：arr[::-1]（切片）或arr.reverse()（原地）。链表是基础数据结构中最重要的一种——理解指针/引用的跳转模型是理解所有链式结构（树、图）的前提。

C++

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:

    vector<int> printListReversingly(ListNode* head) {
        vector<int> res;
        if (!head) return res;
        auto p = head;
        while(p) {
            res.push_back(p->val);
            p=p->next;
        }
        reverse(res.begin(), res.end());
        return res;
    }
};

```

Python

Python solution

NQ098 用两个栈实现队列 AcWing 20

"栈是后进先出（LIFO），队列是先进先出（FIFO）——怎么用两个栈模拟一个队列？"小栋把这个经典面试题抛给了小鲁。小华在纸上画起来："第一个栈负责push（入队），第二个栈负责pop（出队）。当需要pop时，如果第二个栈为空，就把第一个栈的所有元素倒进第二个栈——顺序自然就反过来了。这就是计算机科学中最巧妙的设计模式之一。"

输入

一系列操作指令（push x / pop / empty）。

输出

对pop操作输出弹出的值。

范围

操作数 ≤ 100

输入	输出
push 1	1
push 2	2
pop	3
push 3	empty!
pop	
pop	
empty	

思路 双栈模拟队列：stack1负责push，stack2负责pop。pop时若stack2为空则将stack1全部倒入stack2（翻转顺序）。时间复杂度：每个元素最多入栈2次出栈2次，均摊 $O(1)$ 。这是理解"均摊分析"的经典案例。

技巧 Python用list作栈：append()=push, pop()=pop。两个list分别模拟两个栈。虽然Python有collections.deque，但理解手写实现比调用库更重要——面试常考题。

C++

```

class MyQueue {
public:
    stack<int> stk, cache;
    /** Initialize your data structure here. */
    MyQueue() {

    }

    /** Push element x to the back of queue. */
    void push(int x) {
        stk.push(x);
    }

    void copy(stack<int>& a, stack<int>& b){
        while (a.size()) {
            b.push(a.top());
            a.pop();
        }
    }

    /** Removes the element from in front of queue and re
turns that element. */
    int pop() {
        copy(stk,cache);//拷贝到cache数组
        int res = cache.top();
        cache.pop();
        copy(cache,stk);
        return res;
    }

    /** Get the front element. */
    int peek() {
        copy(stk,cache);//拷贝到cache数组
        int res = cache.top();
        copy(cache,stk);
        return res;
    }

    /** Returns whether the queue is empty. */
    bool empty() {
        return stk.empty();
    }
};

/**
 * Your MyQueue object will be instantiated and called as
such:
 * MyQueue obj = MyQueue();
 * obj.push(x);
 * int param_2 = obj.pop();
 * int param_3 = obj.peek();
 * bool param_4 = obj.empty();
 */

```

Python

Python solution

NQ099 斐波那契数列(类) AcWing 21

"这次我们用面向对象的方式实现斐波那契！"小华说，"定义一个类/结构体，用成员变量存储状态。C++的class或struct都可以——区别是struct默认public，class默认private。Python的class简单直观。注意N=39时斐波那契数已经超过6千万，C++要用long long存储。"

- 输入** 一个整数n。
- 输出** 第n项斐波那契数。
- 范围** $0 \leq n \leq 39$

输入	输出
5	5

思路 用类封装：C++ `class Solution { public: int Fibonacci(int n) {...} }`。递推法 $O(n)$ ， $a, b = b, a + b$ 模式。 $n \leq 39$ 时int够用 ($F(39) \approx 6.3 \times 10^7 < 2^{31}$)。 $n \geq 40$ 需long long。

技巧 Python的类不需要声明成员变量类型——构造函数__init__中直接赋值。C++的struct和class唯一区别是默认访问权限。此题的核心是理解"将算法封装成类的方法"——OO思想的萌芽。

C++

```
class Solution {
public:
    int Fibonacci(int n) {
        if (n <= 1) return n;
        return Fibonacci(n - 1) + Fibonacci(n - 2);
    }
};
```

Python

```
# Python solution
```

NQ100 反转链表 AcWing 35

"把链表反转—— $1 \rightarrow 2 \rightarrow 3 \rightarrow \text{null}$ 变成 $3 \rightarrow 2 \rightarrow 1 \rightarrow \text{null}$ 。"小华在白板上画箭头，"关键操作：把每个节点的next指针指向前一个节点。你需要三个指针——prev（前一个）、curr（当前）、next（下一个，暂存防止断链）。每次迭代：保存下一个→当前指向prev→prev和curr前移。"小鲁试了三遍才理解——"原来指针重新定向这么容易出错，一步顺序都不能乱！"

- 输入** 链表节点值序列，以-1结束。
- 输出** 反转后的链表节点值，每行一个。
- 范围** 链表长度 ≤ 1000

输入	输出
1 2 3 -1	3
	2
	1

思路 迭代反转：prev=null, curr=head; while(curr): next=curr->next; curr->next=prev; prev=curr; curr=next。三指针法——prev(已反转部分)、curr(当前)、next(暂存)。注意顺序：必须先保存next再改curr->next，否则链表就断了。

技巧 反转链表是所有链表操作的基础——合并链表、回文链表、环形链表……都建立在对指针重定向的理解之上。Python虽然没有指针，但用list模拟链表时理解原理同样重要。

C++

```
/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* reverseList(ListNode* head) {
        if (!head || !head->next) return head;

        auto p = head, q = p->next;
        while (q)
        {
            auto o = q->next;
            q->next = p;
            p = q, q = o;
        }
        head->next = NULL;

        return p;
    }
};
```

Python

Python solution

NQ101 合并两个排序链表 AcWing 36

"现在有两个已经从小到大排好序的链表——把它们合并成一个排好序的链表。"小华说，"这就是归并排序中的'归并'步骤。用一个虚拟头节点dummy简化边界处理，然后比较l1和l2的当前值，较小的接入新链表。"小鲁恍然："这就像两叠从小到大排好的扑克牌——每次比较各自最上面那张，小的拿出来放到新牌堆。"

输入 两行，每行链表节点值序列（以-1结束）。

输出 合并后链表节点值，空格分隔。

范围 链表长度 ≤ 1000

输入

1 3 5 -1
2 4 6 -1

输出

1 2 3 4 5 6

思路 双指针归并：dummy头简化逻辑，tail跟随。每次比较l1->val和l2->val，较小的接入tail->next，tail后移。最后把剩余链表接入。时间O(n+m)，空间O(1)。Python用list模拟：while i

技巧 用dummy头节点是链表题的常用技巧——避免处理空链表这种边界情况。合并是归并排序(merge sort)的核心操作——后续会学到完整的归并模板。

C++

```

/**
 * Definition for singly-linked list.
 * struct ListNode {
 *     int val;
 *     ListNode *next;
 *     ListNode(int x) : val(x), next(NULL) {}
 * };
 */
class Solution {
public:
    ListNode* merge(ListNode* l1, ListNode* l2) {
        auto dummy = new ListNode(-1), tail = dummy;
        while (l1 && l2)
            if (l1->val < l2->val)
            {
                tail = tail->next = l1;
                l1 = l1->next;
            }
            else
            {
                tail = tail->next = l2;
                l2 = l2->next;
            }

            if (l1) tail->next = l1;
            if (l2) tail->next = l2;

            return dummy->next;
    }
};

```

Python

Python solution

NQ102 三元组排序 AcWing 862

"给你N个三元组(x,y,z)，按x从小到大排序——这就是结构体排序的经典题。"小华说，"C++中定义一个 struct Triple{int x; double y; string z;}，然后sort+自定义比较函数。Python更优雅——用list of tuples，sorted(arr, key=lambda t: t[0])一行排序。这就是为什么Python在数据处理领域如此流行——一行lambda替代C++十几行代码。"

输入 第一行N，然后N行每行三个值。

输出 按x升序输出三元组（y保留2位小数）。

范围 $1 \leq N \leq 100$

输入

```

3
1 3.5 abc
2 1.2 def
1 2.8 ghi

```

输出

```

1 3.50 abc
1 2.80 ghi
2 1.20 def

```

思路 结构体/元组排序。C++: struct+sort+cmp; Python: sorted(arr, key=lambda x: x[0])。Python的lambda是匿名函数——快速定义比较逻辑而不需要单独定义函数。理解排序的稳定性（x相同时保持原序）。

技巧 Python: `sorted(arr, key=lambda t: t[0])`。lambda是Python函数式编程的利器——它就是一个只能写一行表达式的匿名函数。C++可用lambda: `[](auto &a, auto &b){return a.x`

C++

```
#include <iostream>
#include <algorithm>

using namespace std;

const int N = 10010;

struct data {
    int x;
    double y;
    string z;
    bool operator< (const data & t) const
    {
        return x < t.x;
    }
} a[N];

int main() {
    int n;
    cin >> n;
    for (int i=0; i<n; i++) cin >> a[i].x >> a[i].y >> a[i].z;
    //结构体排序
    sort(a, a+n);
    //输出结果
    for (int i = 0; i < n; i++)
        printf("%d %.2lf %s\n", a[i].x, a[i].y, a[i].z.c_str());
    return 0;
}
```

Python

Python solution

NQ103 绝对值 AcWing 810

"定义一个abs函数——但有一个要求：支持int和double两种类型的参数。"小华说，"这就是函数重载(Overloading)。C++可以写两个同名的abs函数——C++根据参数类型自动选择正确的版本。Python不需要重载——它是动态类型，同一个函数可以接收任何支持<0和-操作的类型。两种语言的设计哲学差异在这里体现得淋漓尽致。"

输入 一个整数或浮点数。

输出 其绝对值。

范围 $-10^9 \leq x \leq 10^9$

输入

-5

输出

5

思路 C++函数重载：int abs(int x)和double abs(double x)可以共存。编译器根据参数类型自动选择。

Python不需要——def abs(x): return -x if x<0 else x就行（鸭子类型）。Python内置abs()已经是多态的。

技巧 C++函数重载的核心：同名不同参。编译器根据参数类型/数量自动匹配。Python通过鸭子类型（duck typing）实现类似效果——只要对象支持所需操作就行。

C++

```
#include <iostream>

using namespace std;

int abs(int x)
{
    if (x > 0) return x;
    return -x;
}

int main()
{
    int x;
    cin >> x;
    cout << abs(x) << endl;

    return 0;
}
```

Python

```
# Python solution
```

NQ104 复制数组 AcWing 814

"写个函数把数组a的内容复制到数组b——但要求用函数封装，且复制后b的内容是a的完全副本。"小华强调，"C++中数组传参会退化为指针——void copy(int a[], int b[], int size)。Python的list直接b=a[:]或b=a.copy()。但注意Python的浅拷贝陷阱——如果list里嵌套了list，需要用copy.deepcopy。"

输入 第一行n和size，第二行n个数。

输出 复制后的数组（空格分隔）。

范围 $1 \leq \text{size} \leq n \leq 1000$

输入	输出
5 3	1 2 3
1 2 3 4 5	

思路 数组复制函数：遍历赋值b[i]=a[i]。C++用for循环或memcpy，Python用切片a[:]（浅拷贝）。理解浅拷贝vs深拷贝——对于int这种不可变对象，浅拷贝安全；对于嵌套list需要用copy.deepcopy。

技巧 Python切片a[:]创建新列表——最常用的复制方式。list.copy()和list(a)也创建浅拷贝。C++的memcpy是原始内存复制——对非POD类型不安全，现代C++推荐std::copy。

C++

```
#include <iostream>
#include <cstring>

using namespace std;

const int N = 110;

void copy(int a[], int b[], int size)
{
    memcpy(b, a, size * 4);
}

int main()
{
    int a[N], b[N];
    int n, m, size;
    cin >> n >> m >> size;
    for (int i = 0; i < n; i++) cin >> a[i];
    for (int i = 0; i < m; i++) cin >> b[i];

    copy(a, b, size);

    for (int i = 0; i < m; i++) cout << b[i] << ' ';
    cout << endl;

    return 0;
}
```

Python

Python solution

NQ105 数组翻转 AcWing 816

"最后一道——把数组的前size个元素翻转。"小栋说。小华点头："这可以验证你对数组操作、函数传参和对称交换的理解。标准做法是从两端向中间交换——arr[0]和arr[size-1]交换，arr[1]和arr[size-2]交换……直到中间相遇。Python用arr[:size][::-1]切片翻转，C++用reverse或手写交换循环。"

输入

第一行n和size，第二行n个数。

输出

翻转前size个元素后的数组。

范围

 $1 \leq \text{size} \leq n \leq 1000$

输入

```
5 3
1 2 3 4 5
```

输出

```
3 2 1 4 5
```

思路 部分翻转：for i in range(size//2): swap(arr[i], arr[size-1-i])。只交换前size个元素，后面的不动。时间 $O(\text{size}/2)$ ，空间 $O(1)$ 。Python用arr[:size]=arr[size-1::-1]结合切片赋值。

技巧 Python切片翻转arr[::-1]最简洁但创建新列表。原地翻转用reverse()或手写交换。理解'对称交换'是理解更多高级翻转（旋转、循环移位）的基础。

C++

```
#include <iostream>

using namespace std;

void reverse(int a[], int size)
{
    for (int i = 0, j = size - 1; i < j; i ++, j -- )
        swap(a[i], a[j]);
}

int main()
{
    int a[1000];
    int n, size;

    cin >> n >> size;
    for (int i = 0; i < n; i ++ ) cin >> a[i];
    reverse(a, size);

    for (int i = 0; i < n; i ++ ) cout << a[i] << ' ';

    return 0;
}
```

Python

Python solution

本章知识点总结

- **链表五操作**：遍历、反向输出、反转、合并、两栈模拟队列——每个都是面试高频题
- **三指针法**：prev/curr/next——反转链表的标准模板，一步顺序都不能乱
- **dummy头节点**：合并链表时用虚拟头简化边界——链表题的通用技巧
- **结构体排序**：C++ struct+sort+cmp vs Python tuple+lambda——一行lambda替代十几行C++
- **函数重载vs鸭子类型**：C++编译期多态 vs Python运行期多态——两种语言哲学的核心差异
- **厦大精神**：自强不息。指针和链表是CS专业的标志——突破这道坎，你就真正进入了计算机科学的世界